

Collective Brain

AI-Powered Team Knowledge Management
Platform

Complete Use Case & Technical Deep Dive

Version 0.3.0

February 2026

Table of Contents

1. Overview
2. Use Cases
3. Technical Architecture
4. Data Ingestion Pipeline
5. AI Query Pipeline
6. Knowledge Graph
7. Insight Engine
8. Room-Scoped Workspaces
9. Database Schema
10. API Reference
11. Frontend Architecture
12. Deployment & Infrastructure
13. Security & Resilience

1. Overview

Collective Brain is an AI-powered team knowledge management platform that ingests your team's documents, code repositories, chat logs, and task data - then lets you ask natural language questions, discover expertise, visualize knowledge relationships, and collaborate in isolated workspaces.

Think of it as: Slack's collaboration + Notion's knowledge base + an AI analyst that understands your entire team's collective knowledge.

Key Capabilities

- **AI-Powered Q&A:** Ask natural language questions about your team's knowledge, get answers with source citations
- **Knowledge Graph:** Three-layer graph (Social + Artifact + Concept) with temporal decay
- **Expertise Discovery:** Find who knows what, identify bus factors and knowledge silos
- **Room Workspaces:** Isolated collaborative spaces with their own data, analytics, and AI context
- **Real-Time Chat:** WebSocket-based room messaging with integrated AI queries
- **Multi-Source Ingestion:** Markdown, Git, Slack, Discord, PDF, DOCX, and task data
- **Automated Insights:** Weekly summaries, pattern detection, risk identification

2. Use Cases

Use Case 1: "Who knows about X?"

A manager needs to assign a Kubernetes migration task. They ask: "Who has the most experience with container orchestration?" The AI searches ingested docs, git commits, and chat logs, then returns members ranked by expertise with source citations showing exactly where their knowledge was demonstrated.

Use Case 2: "What happened last week?"

A team lead returning from vacation asks: "Give me a weekly summary of the team's activity." The Insight Engine analyzes the last 7 days of contributions and generates an LLM-formatted narrative with highlights, key decisions, and recommendations.

Use Case 3: "Are we at risk?"

The system automatically detects organizational patterns:

- **Bus Factor Risk:** Only one person knows the payment system
- **Knowledge Silos:** Backend and frontend teams have zero shared artifacts
- **Stale Expertise:** No one has touched the auth module in 90+ days
- **Strong Collaboration:** Alice and Bob consistently co-contribute to shared artifacts

Use Case 4: Room-Scoped Workspaces

Each team or project gets an isolated room with its own ingested data, knowledge graph, analytics, insights, and AI context. Data never leaks between rooms - like separate Slack workspaces but with full AI-powered knowledge management.

Use Case 5: Real-Time Team Collaboration

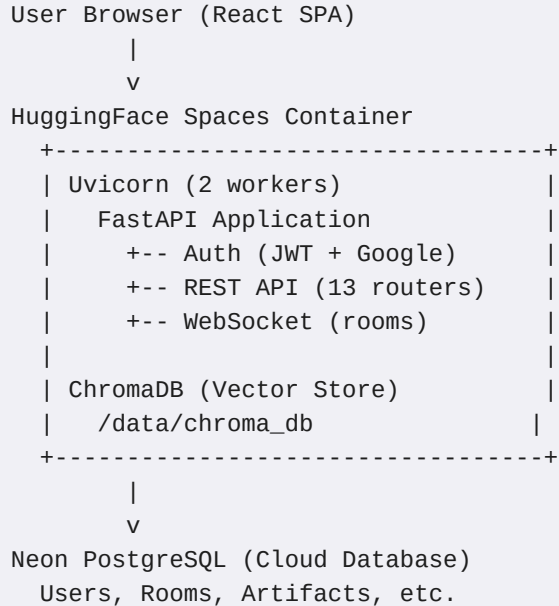
Team members in a room can chat in real-time via WebSocket, ask the AI questions scoped to that room's data, start threaded discussions linked to specific insights or members, and add new members by searching usernames.

3. Technical Architecture

Technology Stack

Layer	Technology
Frontend	React 19 + TypeScript 5.9, Vite 7, Tailwind CSS 4
Backend	FastAPI (Python 3.11), Uvicorn ASGI, 2 workers
Database	PostgreSQL (Neon) / SQLite via SQLAlchemy 2.0
Vector DB	ChromaDB 0.5 with HNSW index, cosine similarity
Embeddings	Sentence-Transformers all-MiniLM-L6-v2 (384-dim)
LLM	Multi-provider: Claude, Mistral (HF), Ollama
Agent	LangGraph ReAct agent with 9 tools OR RAG pipeline
Auth	JWT (30min) + bcrypt + Google OAuth
Real-time	WebSocket per room, optional Redis pub/sub
Visualization	react-force-graph-2d, Recharts
Deployment	Docker multi-stage, HuggingFace Spaces + Neon

System Architecture



4. Data Ingestion Pipeline

Pipeline Flow

```

Source (Markdown, Git, Slack, Discord, PDF, DOCX, Tasks)
  |
  v
Connector (parses format, extracts text + metadata)
  |
  v
Member Resolution (match @mentions, emails, aliases)
  |
  v
Chunking (512 chars, 64 overlap, recursive splitting)
  |
  v
Embedding (all-MiniLM-L6-v2, batch of 64)
  |
  v
Storage:
ArtifactRecord -> PostgreSQL (metadata, room_id)
ContributionRecord -> PostgreSQL (member, topics)
Vector chunks -> ChromaDB (embeddings + metadata)

```

Six Ingestion Connectors

- **Markdown:** Parses .md files with YAML frontmatter, extracts @mentions and topic headers
- **Git:** Mines commit history - author, message, changed files, branch via GitPython
- **Slack:** Chat export JSON parsing with user mapping and thread reconstruction
- **Discord:** Discord export format with role-based member filtering
- **Documents:** PDF (PyMuPDF) and DOCX (python-docx) text extraction with formatting hints
- **Tasks:** GitHub/Jira-style issue data with assignee-to-member mapping

Chunking Strategy

Recursive splitting with configurable separators: paragraph breaks, line breaks, sentence endings, and word boundaries. Default chunk size is 512 characters with 64-character overlap to maintain context across chunk boundaries. Each chunk carries full metadata: source_type, source_ref, author, timestamp, artifact_id, and room_id.

5. AI Query Pipeline

Mode 1: RAG Pipeline (Simple, Fast)

The RAG pipeline follows a 5-step process:

- **1. Intent Classification (rule-based):** Categorizes questions into member_recommendation, pattern_analysis, strategy_generation, or general based on keyword matching
- **2. Retrieval:** Embeds the question using SentenceTransformer, queries ChromaDB with cosine similarity (top-8 chunks), applies optional metadata filters (source types, authors, room_id)
- **3. Context Building:** Formats retrieved chunks with source metadata using intent-specific templates. Includes member expertise summaries for recommendations and contribution data for pattern analysis
- **4. LLM Generation:** Loads last 6 conversation messages for continuity, calls the configured LLM provider with system prompt + context (90-second timeout), falls back to a default response if LLM is unavailable
- **5. Persistence:** Saves user message and assistant response to ConversationRecord, extracts related member names from the response, stores source references with relevance scores

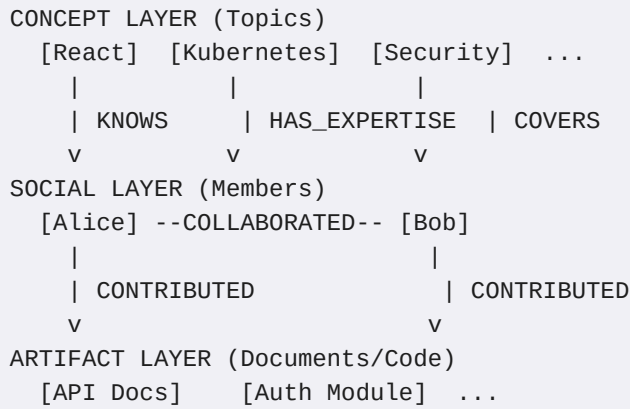
Mode 2: LangGraph Agent (Autonomous, Reasoning)

The LangGraph agent uses a ReAct (Reasoning + Acting) loop with 9 available tools. It autonomously decides which tools to call, reasons about the results, and can chain multiple tool calls to build a comprehensive answer.

Tool	Description
search_knowledge	Semantic search across all ingested data
get_member_profile	Full member expertise + recent contributions
list_team_members	All members with tags and contribution counts
find_experts	Graph-based: who connects to a given topic
get_contributions	Member's activity over N days
analyze_patterns	Bus factors, silos, stale expertise detection
get_topic_network	2-hop topic subgraph visualization
get_weekly_summary	Cached or freshly generated weekly narrative
get_recent_activity	Team-wide activity grouped by type

6. Knowledge Graph

Three-Layer Architecture



Temporal Decay Algorithm

All edges use exponential temporal decay with a 180-day half-life to ensure recent work is weighted more heavily than older contributions:

$$\text{weight} = \exp(-0.693 * \text{age_days} / 180)$$

At 0 days: weight = 1.0, at 180 days: weight = 0.5, at 360 days: weight = 0.25

Contribution Type Weights

Source Type	Weight
Git commits	0.50
Documents (PDF/DOCX)	0.35
Slack / Discord messages	0.30
Markdown files	0.25
Tasks / Issues	0.15

Pattern Detection Algorithms

- **Bus Factor:** Topics with a single contributor are flagged as high risk (confidence: 0.8)
- **Knowledge Silos:** Members with 3+ contributions but zero shared artifacts with others (confidence: 0.7)
- **Strong Collaboration Pairs:** Members sharing 5+ artifacts are flagged as strong collaborators (confidence: 0.6)
- **Stale Expertise:** Topics with no contributions in 90+ days, confidence scaled by temporal decay (0.5-0.9)

7. Insight Engine

Weekly Summary Generation

The engine gathers the last 7 days of contributions, formats them into a structured prompt, and calls the LLM to generate a narrative summary. The output includes highlights (key achievements) and recommendations (action items), extracted via regex parsing. Summaries are cached as InsightRecords with period_start/period_end timestamps.

Pattern Detection Pipeline

Pattern detection runs in two phases:

- **Phase 1 - Graph Analysis (no LLM):** Bus factors, knowledge silos, strong collaboration pairs, and stale expertise are detected using pure graph algorithms against the MemoryGraph
- **Phase 2 - LLM Analysis (optional):** For deeper insights, contribution data is formatted and sent to the LLM for narrative analysis. Results are stored as InsightRecords with confidence scores (0.5-0.9)

Insight Types

Type	Description	Source
pattern	Collaboration and workflow patterns	Graph
risk	Bus factors, stale expertise	Graph
recommendation	Actionable suggestions	LLM
weekly_summary	Periodic team narrative	LLM

8. Room-Scoped Workspaces

Every room is a fully isolated workspace. All data - artifacts, contributions, conversations, insights, discussions, and vector chunks - is tagged with a `room_id` and filtered accordingly. This enables multi-tenant-like isolation without separate databases.

Data Isolation Model

Room A (Engineering)	Room B (Marketing)
+-- Artifacts (code, docs)	+-- Artifacts (campaigns)
+-- Contributions	+-- Contributions
+-- Conversations (AI)	+-- Conversations (AI)
+-- Graph & Analytics	+-- Graph & Analytics
+-- Insights (patterns)	+-- Insights (patterns)
+-- Discussions (threads)	+-- Discussions (threads)
+-- Vector chunks (Chroma)	+-- Vector chunks (Chroma)

Room Features

- **Public Rooms:** Visible in Discover tab, any user can join via self-service
- **Private Rooms:** Invite-only, admin adds members by searching usernames
- **Room Chat:** Real-time WebSocket messaging with user, AI, and system messages
- **Room AI:** Questions are scoped to the room's ingested data via ChromaDB `room_id` filter
- **Room Analytics:** Activity timeline, expertise matrix, topic trends - all room-scoped

Technical Implementation

All 13 backend routers accept an optional `room_id` parameter. When provided, database queries filter by `room_id`, ChromaDB searches use `where:{room_id: 'xxx'}` metadata filtering, and the MemoryGraph constructs room-specific subgraphs. The InsightEngine generates per-room patterns and weekly summaries.

9. Database Schema

The application uses 11 database tables across PostgreSQL (production) or SQLite (development):

Table	Purpose	Key Fields
users	Auth accounts	username, email, password_hash, google_id
members	Team directory	name, aliases, expertise_tags, scores
artifacts	Ingested data	source_type, title, chunk_count, room_id
contributions	Who did what	member_id, artifact_id, topics, room_id
conversations	AI chat sessions	owner_user_id, visibility, room_id
messages	Chat messages	role, content, sources, related_members
insights	Generated patterns	insight_type, confidence, room_id
chat_rooms	Collaborative spaces	name, is_public, message_count
chat_room_members	Room membership	user_id, role (admin/member)
chat_room_messages	Room chat	message_type (user/ai/system)
discussion_threads	Forum threads	context_type, context_id, room_id

Migration System

The application uses a custom migration system (no Alembic). On startup, `init_db()` calls `Base.metadata.create_all()` to create missing tables, then `_run_migrations()` adds missing columns to existing tables via `ALTER TABLE`. This ensures schema evolution without data loss.

10. API Reference

Authentication (4 endpoints)

Method	Path	Description
POST	/auth/register	Create account with username/email/password
POST	/auth/login	JWT token generation (30min expiry)
POST	/auth/google	Google OAuth with auto account creation
POST	/auth/reset-password	Password reset with 6-digit code

Knowledge Ingestion (6 endpoints)

Method	Path	Description
POST	/ingest/markdown-upload	Upload and parse markdown files
POST	/ingest/git	Mine git repository commit history
POST	/ingest/slack	Ingest Slack message exports
POST	/ingest/discord	Ingest Discord message exports
POST	/ingest/documents	Upload PDF/DOCX files
POST	/ingest/tasks	Ingest task/issue tracking data

Query & Conversations (5 endpoints)

Method	Path	Description
POST	/query	AI question-answering (RAG or LangGraph)
GET	/conversations	List user's conversation history
GET	/conversations/{id}	Full conversation with messages
POST	/conversations/{id}/share	Share conversation with users
DELETE	/conversations/{id}	Delete a conversation

Members & Expertise (5 endpoints)

Method	Path	Description
GET	/members	List all team members with expertise
GET	/members/{id}	Member profile with contributions
POST	/members	Create a new member manually
PUT	/members/{id}	Update expertise tags, strengths
PUT	/members/{id}/aliases	Manage name aliases for matching

Knowledge Graph (4 endpoints)

Method	Path	Description
GET	/graph/full	Complete 3-layer graph visualization
GET	/graph/member/{id}	1-hop member subgraph
GET	/graph/topic/{topic}	2-hop topic subgraph
GET	/graph/expertise-matrix	Member x Topic heatmap data

Insights & Analytics (10 endpoints)

Method	Path	Description
GET	/insights/dashboard	Key stats and top insights
GET	/insights/weekly	Cached weekly team summary
GET	/insights/patterns	Detected patterns and risks
POST	/insights/generate	Force pattern detection
GET	/analytics/activity-timeline	Contribution counts over time
GET	/analytics/expertise-matrix	Topic scores per member
GET	/analytics/source-breakdown	Artifact counts by type
GET	/analytics/contribution-types	Contribution type distribution
GET	/analytics/member-activity	Per-member activity ranking
GET	/analytics/topic-trends	Top topics over time period

Rooms & Discussions (10 endpoints)

Method	Path	Description
GET	/rooms	List user's rooms
POST	/rooms	Create new room (public/private)
GET	/rooms/discover	Browse public rooms to join
POST	/rooms/{id}/join	Self-service join for public rooms
POST	/rooms/{id}/messages	Send message to room
POST	/rooms/{id}/ai	Ask AI within room context
WS	/rooms/{id}/ws	WebSocket for real-time chat
POST	/discussions	Create discussion thread
GET	/discussions	List threads with filters
POST	/discussions/{id}/msgs	Add message to thread

11. Frontend Architecture

Technology Stack

- **Framework:** React 19.2 with TypeScript 5.9
- **Build Tool:** Vite 7.3 with @tailwindcss/vite plugin
- **Styling:** Tailwind CSS 4.2 with dark mode support
- **State Management:** @tanstack/react-query 5.90 for server state
- **Routing:** React Router DOM 7.13 with protected routes
- **Animations:** Framer Motion 12.34 for page transitions
- **Icons:** Lucide React 0.575 (500+ icons)
- **Graphs:** react-force-graph-2d 1.29 for knowledge graph visualization
- **Charts:** Recharts 3.7 for analytics (line, bar, heatmap)
- **Auth:** @react-oidc/google 0.13 for Google Sign-In

Page Routes (13 pages)

Route	Page	Description
/login	LoginPage	JWT login + Google Sign-In
/register	RegisterPage	Account creation form
/forgot-password	ForgotPasswordPage	Password recovery flow
/	DashboardPage	Stats, top members, insights
/chat	ChatPage	Conversational AI Q&A
/ingest	IngestPage	Upload documents, git repos
/members	MembersPage	Team explorer with cards
/members/:id	MemberDetailView	Profile, expertise, contributions
/graph	GraphPage	Interactive force-directed graph
/analytics	AnalyticsPage	Timeline, heatmap, trends
/rooms	RoomsPage	My Rooms + Discover tabs
/rooms/:id	RoomChatPage	Real-time chat + AI queries
/discussions	DiscussionsPage	Threaded forum

12. Deployment & Infrastructure

Docker Multi-Stage Build

```
Stage 1: Frontend Build (Node 20-alpine)
  npm ci -> npm run build (Vite)
  Output: /app/dist/

Stage 2: Backend + Static (Python 3.11-slim)
  Install: build-essential, libpq, git
  Install: PyTorch CPU-only (saves 1.5GB)
  Install: requirements.txt
  Copy frontend build -> /app/static/
  CMD: uvicorn app.main:app --workers 2
```

Production Environment (HuggingFace Spaces)

- **Container:** Python 3.11-slim with 2 Uvicorn workers on port 7860
- **Database:** Neon PostgreSQL (free tier, 0.5GB, always-on cloud database)
- **Vector Store:** ChromaDB persistent on /data volume
- **Persistent Storage:** HF Spaces /data volume for ChromaDB data

Key Environment Variables

Variable	Purpose
CB_DATABASE_URL	PostgreSQL connection string (Neon)
CB_LLM_PROVIDER	claude ollama mistral
CB_MISTRAL_API_KEY	HuggingFace Inference API key
CB_CLAUDE_API_KEY	Anthropic API key (if using Claude)
CB_JWT_SECRET	JWT signing secret (change from default!)
CB_GOOGLE_CLIENT_ID	Google OAuth client ID
CB_AGENT_MODE	rag (simple) langgraph (autonomous)
CB_CHROMA_PERSIST_DIR	ChromaDB data directory path
CB_REDIS_URL	Optional Redis for pub/sub and caching

13. Security & Resilience

Authentication & Authorization

- **Password Hashing:** bcrypt with 12 salt rounds via passlib
- **JWT Tokens:** HS256 algorithm, 30-minute expiry, stored in localStorage
- **Google OAuth:** ID token verification via google-auth library, auto account linking
- **Password Reset:** 6-digit codes with 15-minute expiry
- **Rate Limiting:** Auth endpoints: 5 register/min, 10 login/min per IP

Circuit Breaker Pattern

External service calls (LLM, embeddings) are protected by a circuit breaker that prevents cascading failures:

- **CLOSED (normal):** Requests flow through, failures are counted
- **OPEN (after 3 failures):** Returns 503 with Retry-After header, no requests sent
- **HALF-OPEN (after 30s):** Allows test requests, 2 successes close the circuit

Rate Limiting

Token bucket algorithm via Redis (or in-memory fallback). General endpoints: 60 requests/minute per user. AI query endpoints: 10 requests/minute per user. Returns 429 Too Many Requests when exceeded.

Graceful Degradation

- **Redis unavailable:** Falls back to in-memory dict for rate limiting and caching
- **LLM unavailable:** Returns fallback response with available context data
- **Database:** PostgreSQL with connection pooling (10+20 overflow) and pre-ping health checks
- **ChromaDB race condition:** Retry logic (3 attempts, 1s delay) for multi-worker startup

Data Protection

- **Room isolation:** All queries filter by room_id, preventing cross-room data leakage
- **Conversation privacy:** Visibility controls: private, shared, or team-wide
- **CORS:** Configurable allowed origins, credentials enabled
- **SQL injection:** SQLAlchemy ORM with parameterized queries throughout